

Cloud Computing (WDS224 · SCA)

Quelle: WDS224 - Cloud Computing (SCA).zip **Dozent:** Prof. Dr. Andreas Schilling (DHBW Ravensburg) **Aufbau:** Kapitel 1 Grundlagen · Kapitel 2 Virtualisierung, Cloud-Dienstleistungen, Cloud-Bereitstellung · Kapitel 3 Docker **Prüfungsformat (laut Probeklausur):** 60 Minuten · **50 Punkte** · keine Hilfsmittel · 3 Aufgaben: (1) Cloud-Computing-Grundlagen 17 P · (2) Virtualisierung 16 P · (3) Docker 17 P

KAPITEL 1 — GRUNDLAGEN

1. On-Premise vs. Cloud Computing

1.1 On-Premise

Definition: Nutzungsmodell, bei dem der Kunde eine Software **erwirbt** und diese auf **seiner eigenen IT-Infrastruktur** betreibt. Traditionelle Art des Hostings.

- Der Kunde bezahlt **neben der Anschaffung auch Betrieb und Wartung** von Hard- und Software.
- Der Kunde trägt das **Ausfallrisiko** sowie das **Risiko des Livegangs**.
-  **Tiefe Anpassungen** der Hard- und Software möglich (unternehmensspezifische Daten und Prozesse, maßgeschneidert).

1.2 Cloud Computing

Wolken-Analogie: Eine Regenwolke besteht aus einer verdichteten Ansammlung von H₂O-Molekülen, die **in der Außensicht als eine Einheit** wahrgenommen wird — in der Innensicht ein hochkomplexes Zusammenspiel. Genauso ist Cloud Computing eine Ansammlung von IT-Infrastruktur, die nach außen als **eine funktionale Einheit** wahrgenommen wird.

Definition (RedHat 2018): „Clouds are **pools of virtual resources** (such as raw processing power, memory, network, storage, or cloud-based applications) **orchestrated by management and automation software**.“

- Nutzung **virtueller IT-Infrastruktur**, die nicht an einen lokalen Rechner gebunden ist und flexibel konfiguriert werden kann.
-  **Es wird kein Eigentum erworben, sondern nur Nutzungsrechte.**
- Gemäß dem Umfang der virtuellen Ressourcen lassen sich verschiedene **Servicemodelle** unterscheiden.
- Der **Cloud-Anbieter garantiert die Funktionsweise** der Ressourcen und übernimmt sämtliche Wartung.
-  **Begrenzte Möglichkeit zur Anpassung.**

1.3 Cloud verändert Betrieb UND Entwicklung

Bereich	Wirkung
Betriebsaufgaben	Bereitstellung der Applikation durch On-Demand-Service · flexible Anpassung der Laufzeitumgebung (Update, Skalierung, Wartung) · Überwachung (Monitoring)
Bereitstellung neuer Anwendungsfunktionen	Neue Entwurfsmuster (z.B. Microservice) · neue Verarbeitungsmöglichkeiten (Big Data, Realtime) · Bereitstellung fertiger Funktionen (OCR, ML, IAM)

Betroffen sind nicht nur **Technologie** und **Betrieb**, sondern auch **Entwicklungsvorgehen** und **Organisationsstruktur**.

2. ★ Ausgangslage: Probleme des On-Premise-Betriebs

2.1 Einführung neuer Software

Ursache	Wirkung
Hohe Kapitalbindung — hohe Startinvestition in Hardware & Personal	• Lange Bereitstellungszeiten (2–6 Monate) mit mehrstufigen Entscheidungen • Beantragung war oft erste Projektaufgabe bei maximal ungenauen Vorstellungen → meist wird das Doppelte des Worst-Case-Szenarios beantragt • Keine Software-Experimente (Wirtschaftlichkeit nicht im Voraus nachweisbar) → hohe Einstiegsbarriere für Startups • Unterschiedliche Umgebungen für Dev/Int/Prod → Laufzeitfehler werden erst im Betrieb erkannt
Notwendige Auslastung — hohe Auslastung zur Refinanzierung der Investitionskosten	• Geringe Isolation gegen Laufzeitfehler • Desaströse Auswirkungen bei Serverausfall (alle Anwendungen auf einem gemeinsamen Server) • Mehrfachverwendung von DB-Ressourcen

2.2 Betrieb

Ursache	Wirkung
Wartung nur wenn unvermeidbar — Nicht-Verfügbarkeit bei Wartung führt zum Verschieben	• Unsicherheit des Wiederanlaufs • Updates und Patches werden unzureichend gemacht → Ausfall- UND Sicherheitsrisiken steigen • Hersteller-Lock-In, um Kosten der Hardwarebeschaffung und Inkompatibilitätsrisiken zu senken
Verteidigung des Status Quo — Aktualisierungen der Laufzeitumgebung nur, wenn <i>alle</i> Anwendungen sie unterstützen	• Zurückhalten von Innovation und neuen Funktionen (neue Anwendungen müssen auf etablierten Bibliotheken aufbauen) • Ungewünschte Auswirkungen von indirekten Abhängigkeiten • Gegeneinander statt miteinander — Konsensbildung erhöht das Konfliktpotential der Teams

Typische Konfliktauslöser: Updates von Laufzeitumgebungen (JRE, Drittanbieterbibliotheken) · Updates/Austausch von Datenbanksystemen (z.B. DB2) · Aktualisierung des Betriebssystems · Aktualisierung von Konfigurationsdateien.

2.3 ★ Aktualisierung — Schneeflocken und Serverfestungen

Ursache	Wirkung
Schneeflocken-Konfigurationen — manuelle, undokumentierte Änderungen der Serverkonfiguration	• Gefahr von zunächst unerkannten Fehlern • Laufzeitfehler können nicht reproduziert werden • Serverkonfiguration kann nicht kopiert werden → Verteidigung des Status Quo
Manueller Verwaltungsaufwand — Verwaltungssysteme für Hardware sind oft nicht kompatibel	• Parallele Nutzung aufgrund fehlender Integration der Systeme und Abteilungen • Mangelnde Automatisierung systemübergreifender Anwendungsfälle • Langwierige Fehlersuche

🔑 Schneeflocken-Effekt (Martin Fowler)

Manuelle, undokumentierte Änderungen führen dazu, dass **identisch vorgesehene Server Konfigurationsabweichungen aufweisen** — in Anspielung darauf, dass Schneeflocken zwar zunächst gleich aussehen, bei genauerem Hinsehen aber **individuelle Unterschiede** aufweisen.

🔑 Server-Festungen (Klausurfrage 1a!)

Konfigurationen, bei denen die Administratoren **alles Erdenkliche unternehmen, um einen Neustart der Systeme zu verhindern** — weil unklar ist, ob das System danach wieder korrekt anläuft. Sie entstehen durch mutable Infrastructure mit undokumentierten manuellen Änderungen und sind ein Problem, weil sie **Patches, Updates und Reproduzierbarkeit blockieren**.

3. ★ Die sechs Vorteile von Cloud Computing



3.1 Skalierung

Definition: Fähigkeit eines Anwendungssystems, seine **Leistung flexibel zu erhöhen oder zu reduzieren**. Skalierbare Anwendungen können ihre Leistungsfähigkeit durch die Ausstattung mit **virtuellen Ressourcen** erhöhen oder verringern. **Skalierungsfaktor:** gibt an, wie sich die Leistung des Systems bei einer **zusätzlichen Ressourcen-Einheit** verhält. Kann **linear** oder **exponentiell** verlaufen.

- **Linear:** Erhöhung des Speicherplatzes bei einem Dokumentenmanagementsystem → linear proportional mehr Dokumente ablegbar
- **Exponentiell:** Austausch einer Festplatte durch eine SSD → **jeder** Speicherzugriff wird beschleunigt

Das On-Premise-Problem: Es stehen maximal **100 % der Hardwareleistung** zur Verfügung. In Zeiten geringen Bedarfs entsteht **Verlustleistung**; wenn mehr als 100 % benötigt werden, kann dies **nicht** bereitgestellt werden.

Amazon-Beispiel: Über das Jahr betrachtet hochschwankende Hardware-Auslastung des Webshops — massive Spitzenleistungen in der Vorweihnachtszeit und am Black Friday, massive Verlustleistung während des Jahres. **Diese Herausforderung hat unter anderem zum Bau von AWS beigetragen.**

★ Vertikale vs. horizontale Skalierung

	Vertikale Skalierung (Scale-Up)	Horizontale Skalierung (Scale-Out)
Definition	Leistungssteigerung durch Hinzufügen oder Austausch von Hardware (mehr RAM, stärkere CPU)	Leistungssteigerung durch Hinzufügen zusätzlicher Rechner ; Hardwareausstattung pro Rechner bleibt unangetastet
✓	Keine Änderung der Anwendung erforderlich	Deutlich umfassendere Möglichkeiten zur Skalierung (keine Anforderungen an die Hardwarearchitektur) Leistungssteigerungen im Betrieb umsetzbar
✗	System muss gestoppt werden Skalierung nur bis zu einer gewissen Grenze (Rechnerarchitektur)	Anwendung muss horizontale Skalierung unterstützen , ggf. Änderungen erforderlich

3.2 Innovation

Cloud Provider bieten Zugriff auf innovative Funktionen, welche **in Eigenentwicklung enorme Kosten** verursachen würden. Anspruchsvolle Softwareprojekte können deutlich schneller umgesetzt werden.

Funktionsbaustein	Anwendung
Bilderkennung	Bildinhalte auswerten, 3D-Modelle bauen, Personen/Objekte erkennen
Spracherkennung (Speech Service)	Tonaufnahmen auf semantische Zusammenhänge analysieren; Sprachaufnahmen in Befehle samt Parameterbelegung überführen
Texterkennung / Inhaltsmoderation	Kommentare auf Unzulänglichkeiten überprüfen
General AI (z.B. GPT-3)	ad hoc in unbekannten Daten Wirkzusammenhänge identifizieren, die zuvor nicht trainiert wurden

3.3 Globale Verteilung

- Cloud-Provider haben eine **globale IT-Infrastruktur** → weltweites Publikum bedienbar. *Azure betreibt aktuell mehr als 55 Datenzentren weltweit.*
- Zusätzlich **Anbindungen an alle großen Internetknoten** → weitere Latenzreduktion.
- **Unterversorgte Gebiete:** Süd-Amerika, Mittlerer Osten, Russland, Afrika.
- **Warum wichtig?** Minimale **Latenzzeiten** (Dauer der Übermittlung vom Sender zum Empfänger) ermöglichen neue Anwendungsfälle: Virtuelle Kollaboration, **Cloud-Gaming**, **Telemedizin**, Smarte Assistenten; sowie Verbesserungen: Echtzeit-Verkaufsempfehlung, Echtzeit-Tracking.
- Eine vergleichbare Umsetzung mit eigenen Rechenzentren ist für den Großteil aller Unternehmen **undenkbar** (massive Investitions- und Wartungskosten, langfristige Regionsbindung).

3.4 On-Demand

Radikaler Wechsel des Bereitstellungsprozesses: Der Projektverantwortliche konfiguriert über ein **einfaches Webinterface** die benötigten Ressourcen — sie stehen **unmittelbar** zur Verfügung und können **jederzeit angepasst** werden.

Folgen:

- Entfall des langen, mehrstufigen Genehmigungsprozesses
- **Entfall des Bedarfs, Ressourcen zum unsichersten Zeitpunkt abzuschätzen**
- **Prototypen für innovative Funktionen** leicht pilotierbar (auch wenn die Wirtschaftlichkeit noch nicht bewertbar ist)
- Szenarien möglich, in denen Rechenleistung **nur einmal und nie wieder** benötigt wird (z.B. Batch-Konvertierungen) — **keine Alternativbelegung nötig**

3.5 ★ Skaleneffekte

Skaleneffekte = Kostenvorteile durch **sinkende Kosten pro hergestelltem Gut**. Bei Cloud Computing werden die hohen Kosten für Leistung, Netzwerksicherheit und Ausfallsicherheit **auf viele Cloud-Dienstleistungen verteilt** → **Fixkostendegression**.

Der Wirkungskreislauf (dreimal identisch):

Hohe Investition in Leistung/Sicherheit/Ausfallsicherheit
 ↓
 Erschließung neuer Kunden
 ↓
 Reduzierung der Kosten pro Dienst → noch attraktivere Leistungen

Dimension	Konkrete Skaleneffekte
Leistung	High Performance Computing (z.B. von Universitäten) verlagert sich in die Cloud · 384 GB RAM / 96 vCPU für Konsumenten bestellbar · große Reserven für Leistungsspitzen
Sicherheit	DDoS-Schutz bei großen Cloud Providern · Nutzung von „ Red Teams “ zum Aufdecken von Sicherheitslücken · enorme Ressourcen zur Umsetzung von Sicherheitskonzepten
Ausfallsicherheit	Weltweite Daten- und Rechen-Infrastruktur · integrierte Back-Up-Lösungen mit Snapshots · hochverfügbarer Speicher im ganzen Rechenzentrum

3.6 ★ Wirtschaftlichkeit

	On-Premise	Cloud
Investition	Hohes initiales Investment , aktivierbar, lineare Abschreibung (z.B. 5 Jahre)	Massive Reduktion der initialen Kosten
Kostenart	laufende Kosten (Wartung, Upgrades) · nutzungsunabhängige Kosten	anfallende Kosten entsprechen „ Aufwand “ (im Jahr der Entstehung abschreibbar)
Eigentum	Eigentum an Hardware	Kein Eigentum an Hardware
Wartung	Folgeinvestition nach 3–4 Jahren (Upgrades/Teilaustausch)	Wegfall von Wartung & Upgrade
Auslastung	Hohe Opportunitätskosten zu Beginn · Investment in unsichere Auslastung · Leerlauf wird in Kauf genommen	Variabilisierung der Fixkosten · On-Demand-Leistungspakete · Planungsflexibilität · nutzungsabhängige Kosten

🔑 Fixkostenvariabilisierung

Umwandlung von **Fixkosten in variable Kosten**. Der Kunde zahlt nur für jene Hardwareressourcen, die er tatsächlich benötigt. Wichtigster Vorteil: **Planungsflexibilität**, da die Entwicklung einer Anwendung nicht über einen langen Zeitraum abgeschätzt werden muss.

⚠️ **Der Kostenvergleich täuscht:** Cloud Computing hat durch das Nutzungsentgelt **höhere sichtbare Kosten**. Der Kostenvorteil entsteht durch den **Wegfall typischer Umlagekosten** wie Personalaufwand (Personal, Hardware, Training) und den Wegfall des Wartungsaufwands kritischer Anwendungen. ⚠️ **Aber:** Auch beim Cloud Computing lauern verdeckte Kosten — Anpassung bestehender Anwendungen und **Weiterbildungsmaßnahmen**.

📝 Musterrechnung: 1 Server, monatliche Kosten

On-Premise:

Position	Rechnung	Betrag
Hardware	Dell EMC PowerEdge R740 (2,2 GHz, 16 GB, 600 GB, 750 W) $\approx$ 2.214,16 € → Abschreibung über 3 Jahre = 738,00 €/Jahr	61,50 €
Strom	Vollauslastung 750 W $\times$ 24 h $\times$ 30 Tage = 540 kWh Typische Auslastung 30 % → 162 kWh Kühlung + Verlustleistung (Faktor 1,58) → 255,96 kWh $\times$ 0,30 €/kWh	76,79 €
Personal	1.567 Arbeitsstunden/Jahr bei 44.000 € = 28 €/h $\cdot$ 3 h/Monat Wartung (Update/Failover)	84,00 €
Σ Gesamt		222,29 €

Cloud (Azure, West Europe): D4 V4 — 4 vCPU, 16 GB RAM = **313,82 €**

Die On-Premise-Kosten liegen damit ca. **40 % niedriger** als beim Hosting in der Cloud.

📝 Aufgabe 1 mit Musterlösung

a) Welche Aspekte wurden nicht berücksichtigt, die den Aufschlag rechtfertigen?

- **Deutlich kürzere Bereitstellungszeit:** Azure ca. **15 Minuten** vs. On-Premise Beschaffung + Installation ca. **6–10 Tage**
- **Deutlich reduzierte Opportunitätskosten:** Nicht benötigte Rechenleistung muss nicht bereitgestellt werden
- **Flexible Planänderungen:** Konfiguration flexibel ausweitbar/reduzierbar
- **Kein zusätzlicher Personalbedarf** für die Wartung
- **Keine Verantwortung von Downtime** oder Risiken bei der Installation
- **Vorteile bei Gefahrenabwehr und weltweiter Verteilung**

b) 20 Server — welche zusätzliche Infrastruktur?

Position	Kosten	monatlich
2 × Serverschrank (800 € + 200 € Kabel)	1.000 €	27,78 €
Netzwerkswitches	400 €	11,11 €
Loadbalancer	4.000 €	111,11 €
Software zur Hardwareverwaltung	2.000 €	55,56 €
Σ On-Premise	$20 \times 222,29 + 27,78 + 11,11 + 111,11$	4.595,80 €
Σ Azure	20 × D4 V4	6.276,31 €

→ On-Premise ca. **36 % niedriger**. **Nicht berücksichtigt:** Lüftungsanlage · Versicherung · Gebäudesicherheit · Mehraufwand für Wartung · **Redundanz**.

c) Mit SAN (64 GB, über 2 Standorte gespiegelt, 5 €/GB/Monat):

	Speicher	Gesamt
On-Premise	$2 \times 5 \text{ €} \times 64 \text{ GB} = 640 \text{ €}$	$4.595,80 + 1.000 = 5.595,80 \text{ €}$
Azure	ZRS 64 GB = 7,48 €	$6.276,31 + 7,48 = 6.283,80 \text{ €}$

Beim Speicher ist On-Premise ca. 85-mal so teuer wie die Cloud! Nicht berücksichtigt: Erweiterung des Wartungspersonals · Verwaltungssoftware · Stromkosten · Gebäudesicherheit.

4. ★ Die vier Herausforderungen von Cloud Computing

4.1 Kostenmanagement

Empirische Befunde:

- Die Mehrheit der befragten Betriebe liegt **mehr als 23 % über dem Budget** und prognostiziert **+47 % Kostenzuwachs** für das Folgejahr.
- „Up to **58 %** of organizations said that moving to the cloud has been **more expensive than initially thought**." (Weins 2020)
- „**45 %** of respondents believe they are spending too much on cloud computing and **10 %** said they were **100 % over budget**."*
- *„**45 %** of organizations that perform **lift-and-shift [...] without optimization** will be **over-provisioned by as much as 55 %**, and will **overspend by 70 %** during the first 18 months."*

Vier mögliche Ursachen (Brinda 2018):

1. **Der Erfolg der Cloud-Projekte** führt zu einem hohen Bedarf an Cloud-Ressourcen
2. **Attraktivität der Cloud-Ressourcen für Entwickler** (z.B. Texterkennung wird immer häufiger genutzt)
3. **Cloud Computing wird für nicht passende Anwendungsgebiete verwendet** (alte, nicht optimierte Anwendungen → Cloud-Vorteile nicht nutzbar) ⚠ durch Umfragen bestätigt
4. **Nicht mehr benötigte Cloud-Ressourcen werden nicht gelöscht/freigegeben** (nicht deprovisioniert) ⚠ durch Umfragen bestätigt — häufigste Ursache

4.2 Sicherheit

Risiko	Erklärung
Ungewollter Abfluss von Daten	Fehlerhafte Konfiguration von Gruppenrichtlinien und Zugriffsrechten des Cloud-Datenspeichers ermöglicht Dritten direkten Zugriff. In 2018 sind so bis zu 70 Millionen Datensätze abgegriffen worden.
Unautorisierte Nutzung von APIs	Viele Betriebe kämpfen mit Authentifizierung und Autorisierung von APIs . Angreifer können Programmcode abgreifen und geschützte Funktionen ausführen.
Unzureichendes Zugriffsmanagement	Durch Wegfall räumlicher Beschränkungen wirken sich Defizite (nicht gelöschte inaktive Nutzer, unbekannte Adminrollen, veraltete Zugriffsrechte) gravierend aus.
Weltweit verstreute Angreifer	Aufgrund des globalen Zugriffs wirken sich unzureichend gepatchte Betriebssysteme und Bibliotheken deutlich gravierender aus. Im Internet gibt es aktive Scanner , die genau solche Schwachstellen ausnutzen.

4.3 Keine zentrale IT-Governance

Problem	Erklärung
Einhaltung gesetzlicher Vorgaben	Bei weltweiter Verteilung ist zentral, dass Regeln wie die DSGVO eingehalten werden und Daten nur in bestimmten Regionen gespeichert werden. Ohne zentrale Kontrollgremien besteht die Gefahr, dass Entwickler bei der Provisionierung Fehler machen.
Durchsetzung von Qualitätsstandards	Mit dem Kontrollverlust ist es schwer, einheitliche Standards für Daten- und Softwarequalität umzusetzen und abweichende Projekte zu identifizieren.
Umsetzung von Synergien	Durch die Loslösung von Entwicklung und Betrieb müssen Projektverzeichnisse gepflegt werden, damit Lösungen wiederverwendet und nicht mehrfach programmiert werden.

4.4 ★ Fehlendes Know-How

A) Neues Infrastrukturmanagement: Mutable vs. Immutable Infrastructure

	Mutable Infrastructure	Immutable Infrastructure
Charakter	Traditionelle Vorgehensweise	Neue Vorgehensweise
Vorgehen	Infrastruktur wird nach dem Deployment händisch verändert (Update, neue Ports, Backup)	Infrastrukturkonfiguration findet im Code statt
Dokumentation	optional (wird durch Dringlichkeit vernachlässigt)	Es gibt keine nicht codierte Infrastrukturveränderung
Folge	Undokumentierter Zustand, Schneeflocken-Effekt, Serverfestungen	Aufbau von Expertise und Infrastrukturkonfiguration
Werkzeuge	—	Ansible, Puppet, Docker, Kubernetes (K8s)

Der Grundgedanke der Immutable Infrastructure: „Lieber ein System neustarten, das erst **6 Minuten** online ist, als ein System, das seit **6 Jahren** ohne Reboot online ist.“

B) ★ Code-Automatisierung: CI / CD / CD

Warum notwendig? Aufgrund der **horizontalen Skalierung** ist es nicht mehr möglich, auf jeder verwendeten Maschine Code zu testen und manuell ein Deployment vorzunehmen.

Stufe	Definition	Ziel
Continuous Integration	Automatisierte Durchführung statischer Codeanalyse, Buildskripte sowie Unit-, Integrations- und Systemtests nach Codeänderungen	Sicherstellen, dass der Code kompiliert und sich gemäß den Tests verhält. Durch häufige Durchführung werden Inkompatibilitäten aus paralleler Programmierstätigkeit so früh wie möglich identifiziert .
Continuous Delivery	Baut auf CI auf und automatisiert die Auslieferung der Anwendung — den Bau der ausgelieferten Artefakte sowie der benötigten Deploymentskripte (Vorbereitung des Deployments)	Vermeidung typischer Copy-Paste-Fehler beim manuellen Bauen von Artefakten (z.B. falsche Versionsnummer)
Continuous Deployment	Baut auf CD auf und automatisiert die Inbetriebnahme (Deployment) der aktualisierten Softwareversion	Vermeidung manueller Fehler beim Herunterfahren des alten und Starten des neuen Softwarestands

Aufgabe 2 mit Musterlösung — Argumentationsübung

(Orientierung am Basecamp-Blogpost „Why we're leaving the cloud“ von DHH)

Fall 1: Startup „Biotech&Ich“ (10 Mitarbeiter, potentielltes Übernahmeziel) — CRM: Cloud-CRM **Hubspot** vs. On-Premise **Microsoft Dynamics**

 Pro Cloud (Hubspot)	 Pro On-Premise (MS Dynamics)
Vermeidung eines hohen initialen Investments	Kundendaten sind hochsensibel und dürfen Unternehmensgrenzen nicht verlassen
Vermeidung hoher Opportunitätskosten	Vermeidung eines Lock-Ins an einen Cloud-Anbieter
Vermeidung unklarer Planungsgrundlage (wie viele Nutzer in 5 Jahren?)	Nutzung unterliegt keinen Leistungsschwankungen
Vermeidung des Aufbaus einer Betriebsmannschaft samt Know-How	Gestaffelte Preispolitik der Cloud-Lösung vermeidbar
Hohe Verfügbarkeit und Ausfallresistenz	Nutzung auch bei schlechtem/keinem Internetzugang
Nutzer können weltweit verteilt sein	Hohe Anpassung an bestehende Prozesse
Zukünftige Nutzer bekommen schnell Zugriff	Integration in bestehende On-Premise-Systeme
Vereinfachte Zusammenführung mit anderen Unternehmen (bei Übernahme)	

Fall 2: „Advertisement Guru“ (3.000 Beschäftigte in 5 Ländern, weltweit verteilte Kunden, datengetriebene Instagram-Werbeauswertung)

✓ Pro Cloud	✓ Pro On-Premise
Planungssicherheit durch flexibel konfigurierbare virtuelle Ressourcen	Nutzung bestehender IT-Infrastruktur samt Betriebspersonal
Bedarf an schnell skalierbaren virtuellen Ressourcen	Kundendaten können der DSGVO unterliegen
Bedarf an weltweit verteilter Nutzung	Umsetzung einer zentralen IT-Governance
Integration innovativer Programmbibliotheken (z.B. Sentimentanalyse)	Höhere Expertise für Cloud-Nutzung notwendig
Umsetzung von Prototypen mit unklarer Wirtschaftlichkeit	Integration in bestehende Prozesse
Vermeidung des Ausbaus der IT-Abteilung (Hardware und Personal)	
Nutzung von Skaleneffekten für Sicherheit, Ausfall, Leistung	
„ Unendliche “ virtuelle Ressourcen ermöglichen neue Auswertungsmethoden (Big Data)	

KAPITEL 2 — VIRTUALISIERUNG

5. Grundlagen der Virtualisierung

Definition: Virtualisierung bezeichnet die **Entkopplung der Ausführungsumgebung der Software von der physischen Hardware**. Diese Entkopplung kann durch **virtuelle Maschinen** umgesetzt werden, die **abstrakte Repräsentationen der physischen Hardware** sind.

- **Mehrere isolierte Ausführungsumgebungen** gleichzeitig auf derselben Hardware; die Virtualisierungssoftware verhindert Konflikte beim gleichzeitigen Hardwarezugriff.
- **Hardware kann ohne Auswirkung ausgetauscht werden.**
- ⚠️ Trotz Entkopplung wirken sich die **Leistungsgrenzen der physischen Hardware** weiterhin auf die Ausführungsumgebungen aus (CPU, RAM, Speicher) → **vertikale Skalierung erhöht auch die Leistung der VMs.**

5.1 ★ Simulation vs. Emulation vs. Virtualisierung

	Simulation	Emulation	Virtualisierung
Definition	Vollständige Nachbildung eines Systems durch Software	Stellt gleiche Funktionen nach, aber ohne die interne Logik zu verwenden	Dient der Entkopplung von Hard- und Software
Wirkzusammenhänge	werden exakt 1:1 nachgebildet	werden ggf. angepasst (angepasste Programmroutinen, gleicher Output)	angepasste Implementierungen; gleiche Komponentenarchitektur wie das Nachgebildete
Praxiseinsatz	✗ nicht dafür gedacht — dient dem Erkenntnisgewinn	✓ Emulator kann nachgebildetes System ersetzen	✓ Deutlich geringere Performance-Defizite als Emulation; geteilte Nutzung der gleichen Hardware
Beispiele	Flugsimulator, Wetterberechnung, iPhone Simulator	WSL1, JVM, Rosetta 1 & 2 , virtuelle Netzwerkkarte	ESX-i, VirtualBox, Xen, KVM

⚠ **Zentrale Einschränkung der Virtualisierung:** Es ist **nur möglich, einen x86-Prozessor auf einem x86-Prozessor zu virtualisieren** — eine Virtualisierung auf einem ARM-Prozessor ist nicht möglich. (Genau hier ist Emulation notwendig — z.B. Rosetta 2 emuliert x86-Schnittstellen für ARM.)

5.2 ★ Die drei Gründe für Virtualisierung

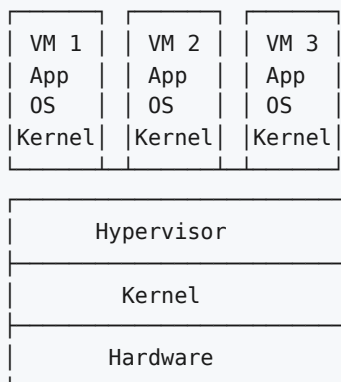
Grund	Konkrete Vorteile
Isolation der Anwendungen	• Eindämmung von Laufzeitfehlern (Sandboxing) — Übergreifen wird verhindert • Einsatz verschiedener Programmiersprachen und Bibliotheken ohne Konsensbildung • Vermeidung ungewollter Seiteneffekte bei Aktualisierungen
Multiplizität von Systemen	• Beschaffung neuer Hardware nicht zwangsweise notwendig (Leistungsreserven nutzen) • Neue Möglichkeiten zum Parallelbetrieb von Alt- und Neu-System • Ausnutzen von Effekten zur Fixkostendegression (Betrieb, Strom, Hardware, Personal, Upgrades)
Portabilität der Systemkonfiguration	• Umzug auf neue Hardware (bei Ausfall, Dev-Prod-Unterschied) • Hardwareänderungen können im Vorfeld geprüft werden • Leichte & schnelle Bereitstellung neuer Umgebungen • Tests mit derselben Konfiguration wie im Produktivbetrieb

5.3 ★ Die Schichtenarchitektur eines Computersystems

App App App	Anwendung: spezifische Funktionalität unter Verwendung der OS-Komponenten
Betriebssystem (OS)	Laufzeitumgebung samt Bibliotheken und grafischen Komponenten
Kernel	Hardwareansprache über Gerätetreiber, Prozess- und Speicherverwaltung, Zugriffsrechte-Kontrolle
Hardware	CPU, RAM, Mainboard, Netzwerk

6. ★ Die drei Arten der Virtualisierung

6.1 Voll-Virtualisierung



- Der **Hypervisor** entkoppelt die Ausführung der VMs von der Hardware, indem er **Ressourcenzugriffe verwaltet und Befehle übersetzt** — er **fängt sämtliche Hardwarebefehle ab und modifiziert sie**, sodass eine VM nur ihr zugewiesenes Kontingent nutzen kann.
- Durch **Hardware-Virtualisierung (z.B. VT-x)** können die Leistungsverluste massiv reduziert werden.
- ★ **Das Gast-OS wird nicht angepasst** und geht davon aus, direkten Hardwarezugriff zu haben.
- Jede VM hat ein **eigenes Image** und **fest zugewiesene Ressourcen**.
- VMs sind **komplett autark** — verschiedene Betriebssysteme möglich (solange prozessorarchitekturkompatibel).

★ Typ-I- vs. Typ-II-Hypervisor

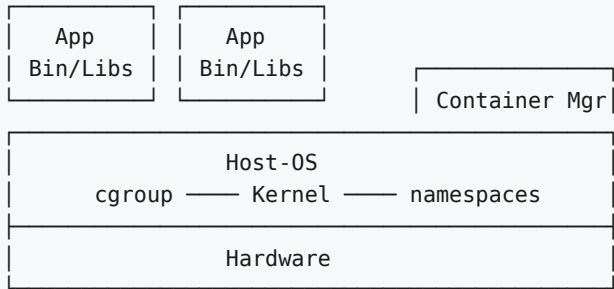
	Bare-Metal Hypervisor (Typ I)	Hosted Hypervisor (Typ II)
Position	sitzt direkt über der Hardware , meist via Web-Interface gesteuert (i.d.R. keine grafische Oberfläche)	inkl. GUI auf einem vollwertigen Host-OS installiert
Performance	✓ Vorteil: Aufrufe gehen direkt zur Hardware — kaum Prozessorausführung, die keiner VM zugeordnet werden kann	✗ Nachteil: Aufrufe werden übersetzt und an das Host-OS weitergegeben
Sicherheit	✓ Vorteil: Kein Gefahrenpotential von anderer Software (Malware) — fokussierter Funktionsumfang	✗ Nachteil: Nebenläufige Anwendungen auf dem Host können Schadsoftware einführen
Sonstiges		✓ Vorteil Multifunktionalität: VMs können zusätzlich zu bestehender Software auf einem Server laufen
Beispiele	VMWare ESX-i, Hyper-V	VirtualBox, VMWare Workstation/Fusion, Parallels

6.2 Para-Virtualisierung

„Para“ stammt aus dem Griechischen und bedeutet „**nebenher**“ — dem Gast-OS ist bekannt, dass es *neben* anderen Betriebssystemen läuft.

- Das **Gast-OS wird angepasst**, sodass es **keine physische Hardware, sondern API-Aufrufe** nutzt, die zum Hypervisor gesendet werden.
- Der Hypervisor verwaltet weiterhin die Ausführung der Prozesse.
- Gerätespezifische Aufrufe: Das Gast-OS nutzt nur einen **virtuellen Treiber**, der an den **nativen Treiber** des Hosts weiterleitet.
- ✓ **Vorteil Performance:** Sehr hohe Performance, da **Hardwarebefehle nicht mehr zur Laufzeit abgefangen** werden müssen.
- ✗ **Nachteil Anpassung:** Anpassungen sind bei **Open-Source-OS möglich, nicht jedoch bei kommerziellen OS** (Windows, macOS).

6.3 ★ Prozessvirtualisierung (Container)



- **Keine vollwertige VM, kein Hypervisor.** Stattdessen werden **Funktionen des Linux-Kernels** eingesetzt, um isolierte Laufzeitumgebungen (**Container**) zu erzeugen.
- Der **Container Manager** konfiguriert die konkrete Ausprägung von cgroups und namespaces.
- ⚠ In Containern können **nur Laufzeitumgebungen** virtualisiert werden, die **denselben Kernel wie das Host-OS** einsetzen (kein Windows/BSD auf Linux-Kernel).
- Ein Container beinhaltet **kein vollwertiges Gast-OS**, sondern **nur die zum Betrieb notwendigen Abhängigkeiten**.
- **Beispiele:** Docker, Cri-o, BSD-Jails

★ Die zwei Kernel-Funktionen

Funktion	Bedeutung
cgroups (control groups)	Einschränkung der NUTZUNG von Ressourcen — feingranulare Steuerung, in welchem Umfang ein Prozess CPU, RAM, Grafikkarte, Dateisystem nutzen darf
namespaces	Einschränkung der SICHTBARKEIT anderer Prozesse/Ressourcen — z.B. dass ein Prozess nur seine eigenen Kindprozesse sehen darf, aber keine benachbarten

- **✓ Vorteil Performance:** Durch Leichtigkeit und **minimalen Overhead** nahezu **native Anwendungsgeschwindigkeit und nativer Speicherbedarf**.
- **✗ Nachteil Isolation:** Trotz Zugriffseinschränkungen **sehen alle Containerprozesse die physische Hardware**. Ferner gibt es **Vorfälle, in denen Angreifer aus Containerprozessen ausbrechen konnten**.

7. ★★ Gegenüberstellung: Voll-Virtualisierung vs. Prozessvirtualisierung

Kriterium	Voll-Virtualisierung (VM)	Prozessvirtualisierung (Container)
Ressourcennutzung	Ressourcen (CPU/RAM/Speicher) werden auch für nicht verwendete Funktionen belegt (reserviert) → ineffizient	Es werden nur jene Ressourcen belegt, die auch tatsächlich benötigt werden
Isolation	Komplette Isolation des Gast-OS vom Host-OS	Gast muss gleichen Kernel wie Host-OS verwenden. Ausbruch aus der Prozessumgebung denkbar und demonstriert
Applikationsstart	Hochfahren des Gast-OS und anschließender Applikationsstart	Direkter Applikationsstart
Ressourcenbereitstellung	Logische Ressourcen (Abstraktionen) werden bereitgestellt oder emuliert; keine spezifischen Treiber nötig	Gast sieht und nutzt die gleichen physischen Ressourcen (keine Abstraktion) ; nutzt Kernel + Treiber des Host-OS
Performance	Hardwareaufrufe müssen abgefangen und angepasst werden → Leistungsverlust 5–10 % (mit VT-x)	Kernel-Aufrufe werden 1:1 ausgeführt → kein Leistungsverlust
Entkopplung	Komplette Entkopplung — unterschiedliche Betriebssysteme der gleichen Prozessorarchitektur möglich	Host und Gast müssen denselben Kernel verwenden
Imagegröße	Umfassendes Image (volles Gast-OS, Abhängigkeiten, Anwendung) ca. 3–4 GB	Leichtgewichtiges Image (nur Abhängigkeiten + Anwendung) ca. 50–500 MB
Horizontale Skalierbarkeit	Eingeschränkt — träger Systemstart, viel nicht benötigter Speicher- und Prozessoroverhead	Sehr gut — sehr geringer Ressourcenbedarf, schnelles Hochfahren
Distributionseinheit	VM	Container

Aufgabe 3 mit Musterlösung

a) Einsatzszenarien

 Do use VM	 Don't use VM
Nutzung der nativen grafischen Oberfläche	Bedarf an flexibel skalierbaren Anwendungen (wg. Overhead)
Voller Funktionsumfang des Betriebssystems benötigt	Einsatz im Entwicklungsprozess umständlich
Starke Isolation der virtualisierten Gäste	Minimierung von Leerlauf
Garantierte Ressourcenverfügbarkeit	Flexible Ressourcenzuteilung
Betrieb monolithischer Anwendungen	Entwicklung & Betrieb skalierbarer Anwendungen
Infrastrukturmanagement durch Sys-Admins	

✓ Do use Container	✗ Don't use Container
Fokus auf Entwicklung und Betrieb einer skalierten Anwendung	Nutzer möchten native grafische Oberfläche verwenden
Bedarf an flexibel skalierbaren Ressourcen	Voller Funktionsumfang des Betriebssystems benötigt
Nur Teile des Betriebssystems werden benötigt	Garantierte Ressourcenverfügbarkeit
Anwendung ausgelegt auf Modularisierung und horizontale Skalierung	Starke Isolation der virtualisierten Gäste
Gleiche Dev/Int/Prod-Umgebungen	Betrieb monolithischer Anwendungen
Entwicklung & Betrieb skalierbarer Anwendungen	Infrastrukturmanagement durch Sys-Admins

b) Vorteile der Kombination von Voll- und Prozessvirtualisierung

- Sicherstellung von **Isolation UND optimaler Ausnutzung** der Ressourcen virtueller Maschinen
- Unterstützung für **traditionelle (monolithische) und moderne** Anwendungen
- **Abstraktion der Hardware**, um gegen Hardwareausfall und Heterogenität robust zu sein
- Kombination der Vorteile für **Betrieb und Entwicklung**
- Umsetzung von „**Software-Defined Infrastructure**“

c) ★ Gründe für Overhead

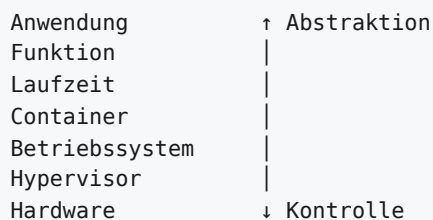
Overhead bei VM durch:	Overhead bei Container durch:
Abfangen und Übersetzen von Hardwarebefehlen	Teilweise Update und Patching von Host-OS und Gast notwendig
Defragmentierungsprozesse bei Host-OS und Gast-OS	<i>(sonst nahezu keiner)</i>
Speicher- und Prozessmanagement bei Host-OS und Gast-OS	
Viele ungenutzte Programme und Hintergrundprozesse beim Gast-OS	
Update und Patching von Host-OS und Gast-OS notwendig	
Virtuelle Ressourcen werden auch bei Nicht-Nutzung reserviert	
Boot-Up des Gast-OS	

d) Vorteile der Prozessvirtualisierung für Entwicklung, Test und Betrieb

- **Gleiche Umgebung für Dev/Int/Prod**
- **Reproduzierbarkeit** von Inkompatibilitäten oder Fehlverhalten
- **Maximale Ausnutzung der Hardware** bei minimalem Overhead
- **Feingranulare Ausnutzung** der Hardware (z.B. **1 VM vs. 50 Container**)
- **Schnelle Boot-Up-Dauer**
- **Reduzierte Pflege** der Ausführungsumgebungen
- **Portabilität** für schnelles Deployment und schnelles **Onboarding von Entwicklern**

8. ★★ Cloud-Dienstleistungsmodelle (Servicemodelle)

8.1 Die 7 Abstraktionsebenen (Klausurfrage 2a!)



★ **Kontrolle und Abstraktion stehen in einem gegensätzlichen Verhältnis.** Je höher die gewählte Abstraktionsebene, desto **geringer die Kontrolle** über die zugrundeliegenden Schichten. ⚠ **Mit der Aufgabe der Kontrolle geht die Gefahr einer stärkeren Bindung an den Anbieter (Lock-In) einher**, da man die **Portabilität** verliert.

8.2 ★ Die Modelle mit der Auto-Analogie

Modell	Cloud-Anbieter stellt bereit	Kunde verantwortet	🚗 Analogie
On-Premise	—	alles	Autokauf — Fahrer · maximale Auswahl · einmalige Kaufsumme · Eigentum · eigene Wartung
IaaS (Infrastructure as a Service)	Hardware + Hypervisor ; logische Hardwareressourcen samt Betriebssystem, SLA-gesicherte Verfügbarkeit , Wartung	Betriebssystem-Konfiguration und alles darüber; kann eigene VMs hochladen	Leasing — Fahrer · moderate Auswahl · monatliche Rate · Fremdeigentum · Fremdwartung
CaaS (Container as a Service)	+ Betriebssystem ; Container-Umgebung; Autoscaling möglich	erstellt das Container-Image ; konfiguriert Ressourcenausstattung (CPU/RAM/Speicher)	Mietauto — Fahrer · kleine Auswahl · On-Demand-Rechnung · Fremdeigentum · ausgelagerte Wartung
PaaS (Platform as a Service)	+ Container + Laufzeit ; Isolierung und Betrieb der Anwendung; frei in der Wahl der Virtualisierung	übergibt nur den Anwendungscode	Taxi — Beifahrer! · minimale Auswahl (Großraum oder regulär) · On-Demand-Abrechnung · Fremdeigentum · ausgelagerte Wartung
FaaS (Function as a Service)	+ Funktion ; elementare Funktionsblöcke (AudioIF, DocumentIF, ImageRecognitionIF)	erstellt nur einen Teil der Endanwendung	Bus (Einzelfahrt) — Beifahrer · keine Auswahl · On-Demand-Abrechnung · Fremdeigentum · ausgelagerte Wartung
SaaS (Software as a Service)	die gesamte Anwendung wird entwickelt und betrieben	nur Konfiguration von Funktionsumfang und Teilen einzelner Funktionen	Bus (Dauerkarte) — Beifahrer · keine Auswahl · monatliche Abrechnung · Fremdeigentum · ausgelagerte Wartung

★ **Der Kontrollverlust hat NICHT nur negative Konsequenzen (Lock-In, Anpassbarkeit) — es werden auch VERPFLICHTUNGEN abgegeben (Failover, Updates, Skalierung, Patching).**

8.3 ★ Zusammenfassungstabelle (Klausurfrage 2b!)

	IaaS	CaaS	PaaS	FaaS	SaaS
Nutzung subjekt	Betriebssystem	Container	Funktion	Anwendung	Anwendung
Abstraktion	Virtualisierung	Betriebssystem	Laufzeit	Funktion	Anwendung
Lock-In	gering	gering (hohe Portabilität)	groß (unterstützte Sprachen/Bibliotheken/Funktionen unterscheiden sich)	sehr groß	maximal (nur ein Anbieter stellt die Anwendung bereit)

Die verwendete Abstraktion ist zentral zur Bestimmung der Verpflichtungen (Patching, Skalierung, Failover) und Freiheiten (Optimierungen, Bibliotheken) des Softwareentwicklers. ⚠
Es gibt keine generelle Empfehlung — die individuellen Bedürfnisse und Kompetenzen des Unternehmens entscheiden. **Es gibt gute Anwendungsfälle für jede Dienstleistungsform.**

Aufgabe 4b mit Musterlösung — Einsatzgebiete je Modell

Modell	Einsatzgebiete	Praxisbeispiele
IaaS	1:1-Migration (Lift-and-Shift ohne Anpassungen, z.B. für vertikale Skalierung) · Hardwarenahe Ausführung (Anwendungen brauchen OS-Anpassungen oder tiefgreifenden Zugriff auf Prozessor-/Speicherstrukturen) · Minimale Latenz (Betrieb in geografischer Nähe zum Endanwender)	GE Oil and Gas: Lift-and-Shift von über 500 Anwendungen zu AWS in 2 Jahren, 30 % Kostenreduktion ohne Optimierungen Novartis: Krebsforschungsplattform, geschätzte Kosten 40 Mio. USD → mit EC2 Spot Instances in 9 Stunden für 4.232 USD
Caas	Hohe Portabilität (keine Abhängigkeiten zu Anbieter oder Sprache) · Entlastung des Betriebs (DevOps: Entwickler übernimmt Betriebsaufgaben und optimiert die Anwendung darauf) · Polyglot Programming (Entwickler nutzen die je Zweck geeignetste Sprache)	PayPal mit Docker EE: +50 % Geschwindigkeit im Build-Test-Deploy-Zyklus; Container-Deployment zu AWS „in a matter of minutes with zero operational involvement “
PaaS	Fokus auf Entwicklung (auch Verbesserungen bei Recruitment, Bindung, Weiterentwicklung) · Minimierung der Time-To-Market (schneller Go-Live) · Zentrale Governance (Programmroutinen prüfen Codestruktur, Qualitätsstandards zentral durchsetzbar)	Scotiabank: Deploymentgeschwindigkeit von 4 Deployments/Jahr auf 3.000 Deployments/Monat Fiserv: Neuschreibung großer Anwendungsteile auf PaaS
FaaS	Schnelle Implementierung (fertige Funktionen ohne Expertise) · Kostenoptimierter Betrieb (pro Aufruf abgerechnet) → keine Über-/Unterprovisionierung, kein bezahlter Leerlauf) · Feingranulare Skalierung einzelner Funktionsblöcke	Coca-Cola: FaaS zum Betrieb digitaler Cola-Maschinen → Infrastrukturkosten um fast 2/3 gesenkt „A container-based solution would probably be cheaper in AWS bills, but our serverless approach is far cheaper from a scalability and monitoring perspective. “
SaaS	Nutzung ohne Anpassung · Schneller Einsatz (keine Installation) · Nutzerbasierte Preise (besonders günstig für kleine/mittlere Unternehmen)	IKEA: innerhalb von 5 Monaten alle Mitarbeiter auf Teams, 75.000 aktive Nutzer , Abschaltung der On-Premises Skype-for-Business-Server GitHub statt Custom Tooling

9. ★★ Cloud-Bereitstellungsmodelle

Bereitstellungsmodelle beschreiben **WO und WIE** die Cloud-Dienstleistungen erbracht werden. Sie unterscheiden sich hinsichtlich **Wartungsaufwand** und **Abstraktion der Hardware**. ⚠ **Zentraler Trade-off:** Starke Kontrolle der Infrastruktur ermöglicht **hohe Datensouveränität**, behindert aber andere Cloud-Vorteile (Skalierung, globale Verteilung). **Es gibt keine „One size fits all“-Empfehlung.**

9.1 Private Cloud

- Einsatz von **Cloud-Software zur automatisierten und standardisierten Bereitstellung** virtueller Ressourcen auf **eigener** Hardware.
- Das **Unternehmen verantwortet Beschaffung, Betrieb und Wartung** der Hard- und Software.
- Betrieb im **lokalen Rechenzentrum** → physischer Zugriff gesichert.
- ★ „**Airgap**“-Umgebungen = Infrastrukturkonfigurationen, die **keinen Internetzugang** haben (bei potenziellen Sicherheitsrisiken).
- **Einsatz bei:** hochsensiblen Daten · eingeschränkter Bandbreite · bestehender Hardware und Personal.

✓	✗
Maximale Kontrolle über Hardwareauswahl	Eingeschränkte Skalierung
Eigene Sicherheitskonzepte	Keine globale Infrastruktur
Nutzung vorhandener IT (Hardware + Personal)	Eingeschränkte Innovation (z.B. keine Spracherkennung)

9.2 Public Cloud

- Nutzung eines Anbieters, der seine Dienstleistungen **der breiten Öffentlichkeit** anbietet (große und kleine Unternehmen, Selbstständige).
- **Klassische Form** des Cloud Computing mit **Abgabe der Verantwortung von Betriebsaufgaben**.
- Realisiert die Vorteile: **Fixkostenvariabilisierung, Skaleneffekte, globale IT, Skalierbarkeit**.
- **Einsatz bei:** Projekten mit **vielen Unsicherheiten, global verteilten Nutzern, innovativen Funktionen**.

✓	✗
Volles Skalierungspotential	Eingeschränkte Kontrolle (sehr geringe Einblicke in Soft-/Hardware)
Globale Infrastruktur	Gefahr eines Lock-In-Effekts (Betrieb wird zunehmend an den Anbieter angepasst)

9.3 Hybrid Cloud

- **Kombinierte Nutzung einer Private Cloud UND einer Public Cloud.**
- Über eine **einheitliche Verwaltungssoftware** (die verschleiert, woher die Ressourcen kommen) können zentrale **Governance- und Sicherheitskonzepte** sowie die Cloud-Vorteile umgesetzt werden.
- **Geringere Lock-In-Gefahr** durch die kombinierte Nutzung.
- **Einsatz bei:** Projekten mit Bedarf an **kurzfristiger Skalierung trotz Schutz sensibler Daten**.

✓	✗
Nutzung vorhandener Infrastruktur	Hohe Komplexität (Konfigurationskomplexität)
Nutzung der Public-Cloud-Vorteile	Hohe Gesamtkosten (Betriebskosten von Private und Public Cloud)

9.4 Multi Cloud

- **Gleichzeitige Nutzung verschiedener Public Clouds** über eine **einheitliche Verwaltungsoberfläche**.
- **Vermeidet Anbieter-Lock-In**, da von den anbieterspezifischen Dienstleistungen abstrahiert wird; zentrale Governance (z.B. Kostenüberblick) und Sicherheitskonzepte umsetzbar.
- **Kostenvorteile**: Für jede Aufgabe kann der **günstigste** Cloud-Anbieter ausgewählt werden.
- **Einsatz bei**: Unternehmen, die **bereits intensiv Cloud-Anbieter einsetzen** und ihren Cloud-Betrieb konsolidieren/optimieren möchten.

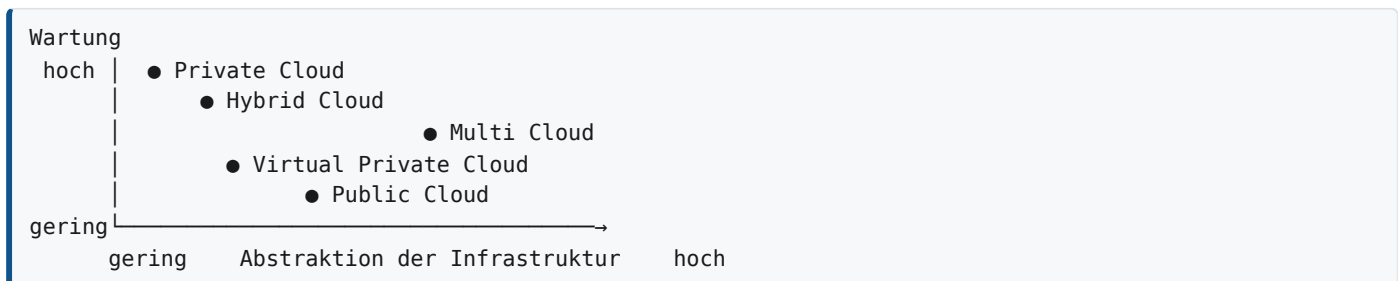
✓	✗
Vermeidung Lock-In	Hohe Gesamtkomplexität
Kostenoptimierter Betrieb	⚠ Verwaltungssoftware an Public Cloud gebunden — Lock-In verlagert sich auf die Verwaltungssoftware

9.5 Virtual Private Cloud (VPC)

- Nutzung der **nicht öffentlich zugänglichen Infrastrukturbereiche einer Public Cloud**.
- **Separate Netzwerkbereiche**, die **nur via VPN** erreicht werden können → größtmögliche Isolation gegenüber anderen Nutzern.
- Erfordert **hohe Expertise in Netzwerksicherheit und Cloud-Konfiguration**.
- **Einsatz bei**: neu aufzubauender IT-Infrastruktur, die die Vorteile des Cloud Computing benötigt.

✓	✗
Volles Skalierungspotential	Abhängigkeit von einem Anbieter (Lock-In via VPN und angepasste Konfiguration)
Globale Infrastruktur	Komplexe Einrichtung

9.6 ★ Einordnung: Wartungsaufwand vs. Abstraktion



- **Private Cloud**: höchster Wartungsaufwand, geringstes Abstraktionsniveau
- **Hybrid Cloud**: geringerer Wartungsaufwand (Teile ausgelagert), höheres Abstraktionsniveau für den Public-Teil
- **Virtual Private Cloud**: geringerer Wartungsaufwand, erhöhter Abstraktionsgrad — aber **spezifische Netzwerk-Expertise** weiterhin erforderlich
- **Public Cloud**: **geringster Wartungsaufwand**, Gefahr des Lock-Ins
- **Multi Cloud**: **erhöhter Wartungsaufwand**, aber durch standardisierte Ressourcenausgestaltung über alle Anbieter **am abstraktesten** von der konkreten Infrastruktur

Aufgabe 4c mit Musterlösung — Cloud-Migrationspfade

Ausgangslage	Start	Zwischenziel	Endziel
Bestehende Hard- und Software	Private Cloud	Hybrid Cloud	Multi Cloud
Keine bestehende Hard- und Software	Public Cloud	—	Multi Cloud (zur Vermeidung des Anbieter-Lock-Ins)
Bestehende Software	Virtual Private Cloud	teilweise Integration von Public-Cloud-Ressourcen	Multi Cloud

10. Edge Computing

Definition: Verarbeitung von Daten **so nahe wie möglich am Entstehungsort**.

Far Edge (Producers) → Edge Computing → Private/Public Cloud
 Edge Devices ortsabhängige Auswertung
 erzeugen + nutzen Entscheidungen
 Daten direkt

Beispiel Gießerei: Videokamera zur Qualitätskontrolle (Far Edge) → automatische Bildauswertung justiert Produktionsparameter des Gießvorgangs (Edge) → Daten zur Auswertung in die Cloud.

- **Großes Wachstum der datenproduzierenden Geräte** erwartet.
- Durch Edge Computing müssen **nicht alle Daten in einer Cloud verarbeitet werden** → Vorteile für **Datensicherheit und Latenz**.
- **✗ Herausforderungen: Wartung** (wie erreicht man Software-Updates bis zu den Datenproduzenten?) und **Heterogenität** der Daten und Geräte.

★ Anwendungsfelder von Edge Computing

Feld	Beispiel
Unabhängige Verarbeitung (Ausfall-Resilienz)	Verarbeitung unabhängig von der Internetverbindung — wichtig für kritische Infrastruktur
Datenschutz (DSGVO)-konforme Speicherung	Sensible Daten direkt am Entstehungsort speichern
Vermeidung sensibler Daten	Sensible Daten werden gar nicht erst gespeichert — z.B. ein Security-Gateway , das nur die Zugangsschlüssel kennt und nicht jede Anfrage zentral prüft (Authentifizierung)
Extremer Bandbreitenbedarf	Live-Event, Analyse von Turbinendaten
Datenreduktion	Datenkompression gleicht reduzierte Bandbreite aus (Ü-Wagen am Point-of-Interest)
Echtzeitverarbeitung	Smart Factory — am Fließband fehlerhafte Teile identifizieren und herausfiltern

Aufgabe 4d mit Musterlösung — Edge Computing vs. Private Cloud

Kriterium	Edge Computing	Private Cloud
Betrieb	Nahe am Ort, an dem die Daten produziert werden	Unabhängig vom Entstehungsort der Daten
Anwendungsfokus	Klarer Bezug zu den Anwendungsszenarien	Nutzung unterschiedlicher Virtualisierungsformen , um diverse Anwendungsgebiete umzusetzen
Wartung	Individuell zugeordnete Aufgaben zur Wartung und zur Kompatibilität heterogener Datenproduzenten	Nutzung der Dienstleistungsformen zur klaren Zuständigkeitsaufteilung; Virtualisierung managt Heterogenität
Verarbeitungsleistung	Eingeschränkt, um die Nähe zu den Produzenten umzusetzen	Ziel ist hohe Verarbeitungsleistung für rechenintensive Einsatzgebiete
Verarbeitungsstrategie	Dezentral	Zentral
Einsatzgebiet	Anwendungen, die minimale Latenzen benötigen oder Daten vorverarbeiten	Flexible Skalierung von Anwendungen und schnelle Bereitstellung von Ressourcen

Aufgabe 4a — Stellungnahme zum Sam-Newman-Tweet

Der Punkt: Durch die Nutzung der unterschiedlichen Virtualisierungsschichten ist es **in Summe aufwändiger, alle Schichten auf dem aktuellen Patchlevel zu halten**. Beim traditionellen On-Premise-Betrieb muss „nur“ **Hardware, Betriebssystem und Software** aktuell gehalten werden.

Die Gegenargumente: Die Unterscheidung der Virtualisierungsschichten ermöglicht es, **standardisiert festzulegen, was die Aufgabe des Nutzers und was die Aufgabe des Cloud-Betreibers ist**. Nutzer können sich auf ihre **Kompetenzen fokussieren**. Ferner geht es darum, den Nutzern **größtmögliche Flexibilität** zu bieten und dennoch **Gemeinsamkeiten einheitlich zu patchen**.

KAPITEL 3 — DOCKER

11. Grundlagen

11.1 Das Container-Paradigma (Frachtschiffahrt)

Vor dem Standardcontainer	Nach dem Standardcontainer
Manueller Aufwand: Hafenarbeiter betreten das Schiff und entladen die Fracht händisch	Standardisierung: 90 % aller Frachtcontainer sind Standardcontainer — die gesamte Infrastruktur ist darauf optimiert
Gefahr: Kein sicheres Arbeitsumfeld	Sicherheit: Massive Reduktion von Frachtschäden und Diebstählen
Performance: Hafenbrücken werden zum Flaschenhals	Performance: Sehr kurze Dauer für Ladung und Entladung durch Infrastruktur
Unsichere Dauer: Keine Planung, da abhängig von der Verfügbarkeit der Arbeiter	Planung: Standardisierte Infrastruktur gibt Planungssicherheit für Dauer und Kosten

 **Die Zahlen:** Vor Einführung benötigte eine **18-köpfige Crew** eine **8-Stunden-Schicht**, um **80 Tonnen** zu entladen. Nach Einführung entlud eine **9-köpfige Crew** in derselben Zeit **2.000 Tonnen**.

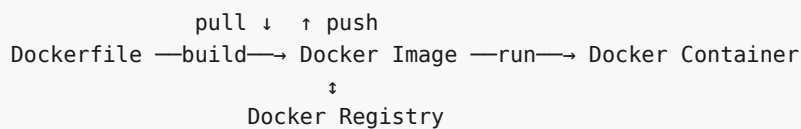
11.2 Was ist Docker?

- Ein Werkzeug, mit dem Anwendungen **gebaut, verteilt und betrieben** werden können.
- **De-facto-Standard zur Prozessvirtualisierung** unter Nutzung der Kernelfunktionen **cgroups und namespaces**.
- **Kostenlos**; Code der Ausführungsumgebung und des Speicherformats teilweise offengelegt → große Verbreitung.
- Legte — ähnlich zum Container in der Schifffahrt — **den Grundstein für weitere Werkzeuge** und Umsetzungen des Cloud Computing.
- **Programmiersprachen-agnostisch**: Es wird ein **standardisiertes und portables Abbild (Image)** der Anwendung und der notwendigen Systemkonfiguration erzeugt.

Zwei Bestandteile:

1. **Docker-Client** — Kommandozeilenprogramm, das eingegebene Befehle übersetzt und via **REST-Schnittstelle** an den Docker-Daemon überträgt
2. **Docker-Daemon (Server)** — verantwortlich für **Erstellung und Parametrisierung der Linux-Container** sowie die Überführung der portablen Abbilder in ausführbare Einheiten

11.3 ★ Der Docker-Workflow



Merkhilfe: Ein **Docker Image** ist wie ein **Kuchenrezept**. Anhand des Rezepts lassen sich **ein oder mehrere Kuchen** parallel oder nacheinander erstellen (**Container**). Das Rezept kann über eine Internetplattform (**Docker Registry**) verteilt und bezogen werden. Und es lässt sich in eigener kreativer Arbeit verfassen (**Dockerfile**).

12. ★ Docker Container

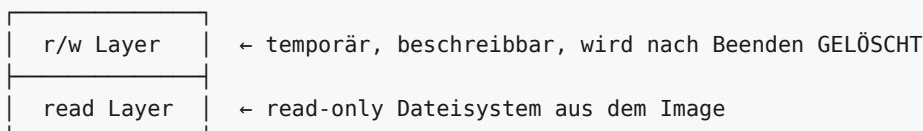
12.1 Image vs. Container

Docker Image	Docker Container
Read-Only — nicht veränderbar	Laufzeit-Repräsentation
Beschreibung der Laufzeitumgebung und des auszuführenden Prozesses	Leichte Replikation (mehrere Container aus demselben Image)
Portabel	Keine Hardwarebindung
wird nicht direkt ausgeführt	eine Kopie des Images wird ausgeführt

Alles, was in einem Container passiert, beeinflusst NICHT das Docker Image oder andere Container — alle Änderungen im Container sind nicht permanent. Starker Kontrast zur VM: Dort wird die Abbilddatei **selbst ausgeführt**, sodass alle Laufzeitänderungen **dauerhaft im Image** gespeichert werden.

12.2 ★ Copy-on-Write

Docker Container



Prinzip: Da das gesamte Dateisystem des Containers **nur lesbar** ist, können keine Änderungen vorgenommen werden. Damit dennoch Dateiänderungen möglich sind, werden Dateien **zur Veränderung in die temporär beschreibbare Schicht KOPIERT** und dort verändert — daher der Name **copy-on-write**. Die temporäre Schicht wird **nach Beenden des Containers gelöscht**.

⚠ **Wichtige Präzisierung (aus Musterlösung Aufgabe 5b):** Solange der Container nur **gestoppt** wurde, ist eine erstellte Datei bei einem **Wiederstart** noch da. Wird der Container jedoch **gelöscht**, sind die Datei und sämtliche Änderungen weg.

12.3 ★ Befehlssyntax

docker	container	run	-d	hello-world
↑	↑	↑	↑	↑
Docker Client	Objekt-klasse	Befehl	optionale Parameter	Objektinstanz (Image oder Container)

💡 Bei **ContainerIds** reicht es in der Regel, die **ersten drei Zeichen** zu verwenden — der Docker Daemon löst den Rest automatisch auf.

12.4 ★ Container-Befehle

Befehl	Wirkung
<code>docker container run hello-world</code>	Erstellt einen neuen Container für das Image und führt ihn aus
<code>docker container start 684</code>	Startet einen bereits existierenden, gestoppten Container ⚠ <i>kann keinen neuen Container starten</i>
<code>docker container stop 684</code>	Stoppt den Container mit der Id, die mit „684“ beginnt
<code>docker container rm 684</code>	Löscht den Container
<code>docker container list</code>	Listet alle laufenden Container samt Status auf
<code>docker container list -a</code>	Listet auch gestoppte Container auf
<code>docker container stats 684</code>	Gibt Statistiken zur Ressourcennutzung aus (CPU/RAM/Speicher)
<code>docker container logs 684</code>	Gibt den Kommandozeilenoutput des Containers aus

12.5 ★ Run-Optionen

Option	Bedeutung
<code>-p</code> <code>80:80</code>	publish — leitet Netzwerkverkehr des Hosts zu einem Container um. Notation: <code>-p <HostPort>:<ContainerPort></code>
<code>-d</code>	detach — Container läuft im Hintergrund weiter (statt im <i>attached mode</i> , in dem die Kommandozeile blockiert ist); gibt die Id des Containers zurück
<code>-it</code>	interactive + tty — ersetzt die Startkommandos des Containers durch den mitgegebenen Befehl und gibt eine interaktive Befehlseingabe im Container
<code>--rm</code>	Löscht gestoppte Container sofort → vermeidet brachliegende Container
<code>--env</code>	Setzt Umgebungsvariablen zur Laufzeit
<code>--env-file</code>	Bindet eine Datei mit Umgebungsvariablen zur Laufzeit ein



Hands-On: Was passiert bei `docker container run hello-world`?

1. Docker Client ruft Docker Daemon auf
2. Docker Daemon prüft, ob das benötigte Image LOKAL vorliegt
3. Docker Daemon kontaktiert die Docker Registry und sucht dort nach dem Image
4. Docker Daemon lädt das Image von der Registry
5. Docker Daemon erstellt einen Docker Container basierend auf dem Image
6. Docker Daemon gibt den Output des Containers auf der Befehlszeile aus
7. Docker Daemon stoppt den Container



Aufgabe 5 mit Musterlösung

```
# a) tomcat detached, Hostport 80 → Containerport 8080
docker container run -d -p 80:8080 tomcat
docker container list          # ContainerId identifizieren
docker container stats 935     # Ressourcenlast anzeigen
docker container logs 935     # Logs ausgeben

# b) ubuntu interaktiv, Datei erstellen, Persistenz prüfen
docker container run -it ubuntu
  mkdir test
  cd test
  touch test.txt
  exit                          # stoppt den Container
docker container list -a       # laufende UND gestoppte Container
docker container start -i f0a  # interaktiv wieder starten → Datei ist noch da!
```

c) Was bewirkt `--rm`? Vermeidet **brachliegende, gestoppte Container** — gestoppte Container werden **sofort gelöscht**.

13. ★ Docker Images

13.1 Typische Deployment-Fehlerursachen (Status Quo)

- **Unterschiede in Applikationsversionen** (OS, DB, Laufzeit, Webserver)
- **Unterschiede in der Konfiguration** (Umgebungsvariablen, Skripte, Dateien, Rechte, Zertifikate)
- **Unterschiede in der Herangehensweise** zur Einspielung des Updates
- **Vage Dokumentation** zum Vorgehen oder kein Vorwissen zu wichtigen Konventionen
- **Fehlerhafte Übertragung** der Dateien und Konfigurationen auf die Prod-Umgebung

13.2 ★ Die Schichtenarchitektur

Tag: cloud-comp:1.1

ENV stage prod	← Schicht 3 (+ Prüfsumme)
RUN apt-get install vi	← Schicht 2 (+ Prüfsumme)
FROM Ubuntu	← Schicht 1 (Baseimage)

- **Jede Änderung und Konfiguration wird in einer eigenen Schicht samt Prüfsumme erfasst.**
- Gemäß **immutable infrastructure** ist jede Schicht **unveränderbar** — sie wird nicht verändert, sondern **stets neu gebaut**.
- Anzeige: `docker image history <tag>`

★ Vorteile der Schichtenarchitektur:

1. **Jede Schicht wird nur einmal gespeichert** → keine redundante Speicherung. Images, die sich bis auf eine Änderung am Ende gleichen, belegen **nur einmal Speicher**.
2. Beim Bauen werden die **Prüfsummen jeder Schicht in aufsteigender Weise auf Änderungen überprüft (Build-Cache)**.
3. **Optimierter Down-/Upload** — es werden nur die Schichten geladen, die **nicht bereits auf der Zielmaschine existieren**.
4. ⚠ **Bei Änderung einer Schicht wird diese UND ALLE DARAUF BASIERENDEN Schichten neu gebaut.**

13.3 ★ Tags

```
https://hub.docker.com / plexinc / pms-docker : latest
      ↑           ↑           ↑           ↑
  1. Registry    2. User/Org  3. ImageName  4. Version
  (optional)    (Pflicht)   (Pflicht)   (optional)
```

⚠ **Wird kein Versionstag gesetzt, wird automatisch latest verwendet. Entgegen der wörtlichen Übersetzung steht latest NICHT für die aktuelle Version, sondern nur als Standardwert!** Es ist auch möglich, dass ein **alter** Versionsstand den Zusatz latest trägt.

Tags sind Verweise auf ein konkretes Docker Image. **Unterschiedliche Tags auf dasselbe Image verbrauchen keinen zusätzlichen Speicherplatz.**

13.4 Docker Registry

- **Zentraler Ort zum Speichern und Verteilen** von Docker Images → „**image from the shelf**“-Paradigma.
- Datentransfer basiert auf der Schichtenarchitektur → **nur fehlende Schichten** werden übertragen.
- **public registry** (öffentlich) vs. **private registry** (geschlossener Teilnehmerkreis, z.B. in Unternehmen).
- **Bekannteste public Registry: <https://hub.docker.com>**
 - Ist als **Standard nach der Installation** gesetzt
 - Docker unterhält ein **eigenes Team** zur Aktualisierung der offiziellen Images (MySQL, PostgreSQL, Spring, ...)
 - ★ **Offizielle Images tragen KEINEN Usernamen im Tag**

13.5 ★ Das Dockerfile

Eine **einfache Textdatei** mit dem Namen **Dockerfile**, **ohne Dateiendung**. Build-Befehl: `docker image build -t imagetag .` **Jede Zeile ist eine eigene Instruktion, die eine Schicht erstellt.**

Dreiteiliger Aufbau:

```
FROM openjdk:8-jdk-alpine      ← 1. BASEIMAGE
ENV demo.version=0.9          |
RUN mkdir app                  | 2. Anpassen von Abhängigkeiten
WORKDIR app/                   |   & Konfiguration
COPY target/*.jar app.jar      |
EXPOSE 8080                    |
CMD ["java", "-jar", "app.jar"] ← 3. EINSTIEGSKOMMANDO
```

13.6 ★★ Die Dockerfile-Befehle

Befehl	Funktion	Wichtige Hinweise
FROM	Referenziert das Baseimage , das angepasst wird	⚠ Kein automatisches Update von Bibliotheken — Aktualisierung erfolgt manuell über den Austausch des Tags . Sehr detaillierte Tags verwenden.
ENV	Setzt Startwerte relevanter Umgebungsvariablen	Anpassung des Laufzeitverhaltens . Kann zur Laufzeit überschrieben werden . ⚠ Nicht für sensible Daten!
ARG	Umgebungsvariable nur zur BAUZEIT	Zu verwenden, wenn die Variable keinen Einfluss auf das Laufzeitverhalten hat
RUN	Führt ein Kommando zur Bauzeit aus	Kommandos müssen vom Baseimage unterstützt werden. Anwendungen oft über den Paketmanager installieren.
WORKDIR	Setzt im Container das Ausgangsverzeichnis für alle folgenden Kommandos	Vermeidet die Wiederholung gleicher Verzeichnispräfixe und Copy-Paste-Flüchtigkeitsfehler
COPY	Kopiert Dateien vom lokalen Verzeichnis in das Image	Unterstützt Wildcards ⚠ vorsichtig einsetzen!
EXPOSE	Gibt eingehende Ports für den Container frei	⚠ Unabhängig von der publish -Option! Aus Sicherheitsgründen sind eingehende Verbindungen im Grundzustand untersagt ; EXPOSE erlaubt sie überhaupt erst. <code>-p</code> definiert das Mapping von Host- auf Containerports.
USER	Führt folgende Befehle unter einem unprivilegierten Benutzer aus	⚠ Neue Benutzer gehören standardmäßig der Gruppe root an → manuell eine andere Gruppe anlegen!
CMD	Startet den Applikationsprozess — erster Befehl nach dem Start eines Containers	Kann zur Laufzeit überschrieben werden . Zwei Formen (s.u.)

★★ CMD: Exec form vs. Shell form

```

CMD ["java","-jar","app.jar"]    # ✅ Exec form
CMD java -jar app.jar             # ⚠ Shell form – Vorsicht!

```

Die beiden Notationsformen sind NICHT gleichbedeutend:

- **Exec-Notation:** Der Befehl wird als **eigenständiger Prozess** ausgeführt
- **Shell-Notation:** Der Befehl ist ein **Unterprozess der Shell**

⚠ **Sicherheitsfolge:** Bricht ein Angreifer aus dem Anwendungsprozess aus, hätte er bei der **Exec-Notation keine weiteren Hilfsmittel**, während er bei der **Shell-Notation alle von der Shell angebotenen Werkzeuge** zur Verfügung hätte.

13.7 ★ Image-Befehle

Befehl	Wirkung
<code>docker image list</code>	Listet alle lokal vorhandenen Docker Images auf
<code>docker image history hello-world</code>	Zeigt die einzelnen Schichten des Images
<code>docker image tag alpine v1</code>	Erstellt den neuen Tag <code>v1</code> ⚠ <i>der alte Tag wird nicht gelöscht</i>
<code>docker image pull alpine</code>	Lädt das Image von der Docker Registry herunter
<code>docker image build -t test .</code>	Erstellt ein neues Image mit dem Tag <code>test</code> auf Grundlage des Dockerfiles im gleichen Verzeichnis
<code>docker image rm test</code>	Löscht das Docker Image

13.8 Spring Boot & Docker

- **Umgebungsvariablen** dienen zur: Nutzung unterschiedlicher **Ressourcen und Endpunkte** · **Vermeidung sensibler Daten** (z.B. in dauerhaft gespeicherten Logdateien) · **Beschränkung des Funktionsumfangs**
- **Profile** steuern die **Sichtbarkeit ganzer Klassen** sowie deren Konfiguration. Die Properties-Datei `application-{profile}.properties` wird gemäß dem aktuellen Profil gewählt.
- Die Umgebungsvariable `spring.profiles.active` definiert das aktive Profil.
- `mvn clean install` erzeugt ein **self-contained (fat) Jar**, das alle internen und externen Abhängigkeiten integriert.

14. ★★ Docker Best Practices

14.1 Baseimage

- Das Baseimage sollte **nur die wirklich benötigte Funktionalität** besitzen. Ein geringerer Funktionsumfang reduziert **Downloaddauer, Speicherbedarf und Angriffsfläche**:

Distribution	Größe
<code>ubuntu</code>	~ 188 MB
<code>opensuse:latest</code>	~ 82 MB
<code>alpine</code>	~ 5 MB ★ (35× kleiner als ubuntu)

- **Offizielle Images bevorzugen** — bei diesen sichert Docker Aktualisierungen und Authentizität.
- **Das Baseimage sollte mit einem Tag referenziert werden, NIEMALS mit `latest`.**
- ⚠ **Images sollten stets bei Aktualisierungen des Baseimage NEU GEBAUT werden.** Nur so bleiben die Container dauerhaft gepatcht — **alle Aktualisierungen zur Laufzeit werden beim Neustart des Containers nichtig!**

14.2 ★ Tagging

Warum `latest` strikt zu vermeiden ist:

1. Es können **verschiedene Versionen in der Registry und auf dem Client** referenziert werden
2. Nicht die „aktuelle“, sondern die „**unbekannte**“ **Version** ist gemeint
3. **Keine Charakterisierung** des Image erfolgt
4. **Kein Rücksprung auf vorherige Versionen** möglich

Tag-Typ	Verwendung
Stable Tags (1, 1.2, 1.2.1)	für Abhängigkeiten / Baseimages — um die neuesten Änderungen zu bekommen; über das Schema steuerbar, ob nur Sicherheitsaktualisierungen oder auch Featureupdates berücksichtigt werden
Unique Tags	für eigen erstellte Images — vermeiden ungewolltes Caching und melden Updatebedarf an. Integration der BuildId oder des Datums ist sinnvoll. ⚠ Vorsicht bei Git-Tags , da sie Aktualisierungen der Baseimages nicht abbilden

⚠ **Das Problem doppelter Tags:** Wenn Docker-Container, Orchestrierung, Docker-Registry und Docker-Image alle `test:1.1` referenzieren, kann der Orchestrator **nicht erkennen, dass sich der Inhalt geändert hat** — es werden veraltete Versionsstände ausgeliefert.

14.3 ★ Umgebungsvariablen

⚠ **Umgebungsvariablen dürfen im Dockerfile NICHT für sensible Daten wie Passwörter verwendet werden, da diese mit dem Befehl `docker image history` ausgelesen werden können!**

- Stattdessen sensible Daten **zur Laufzeit via `--env`** einbinden (vgl. MySQL-Image). **Besser: verschlüsselte Varianten nutzen.**
- Bei vielen Laufzeit-Umgebungsvariablen: Auslagerung in eine separate Datei, eingebunden via `--env-file`.
- Bei Variablen, die **nur zum Bau** verwendet werden: **ARG statt ENV**.

★ Lesereihenfolge (spätere überschreiben frühere)

Dockerfile → env-file → Laufzeitargument

14.4 ★ Schichtenarchitektur optimieren

Regel: Je weiter oben ein Befehl steht, desto seltener sollte er sich ändern.

- Bei Abweichungen der Prüfsummen wird die entsprechende Schicht **und alle darunter liegenden** neu gebaut
- Schichten können nur wiederverwendet werden, wenn sie die **gleiche Prüfsumme** aufweisen

```
# ✗ FALSCH – COPY steht zu weit oben
FROM openjdk:8-jdk-alpine      ← Änderungsfrequenz gering
COPY *.jar app/app.jar        ← ⚠ ändert sich bei JEDEM Compiliervorgang!
ENV stage=prod                → macht die Vorteile der Schichtenbildung zunichte
RUN mkdir app
WORKDIR target/
CMD ["java", "-jar", "/app.jar"] ← Änderungsfrequenz hoch
```

Weitere Regeln:

- **Zusammengehörige RUN -Befehle mit && kombinieren**, um unnötige Schichten zu vermeiden und die Bauzeit zu verkürzen
- Befehle **so präzise wie möglich** formulieren ⚠ **insbesondere bei Wildcards** — Änderungen an unnötigen, aber mitkopierten Dateien stoßen einen Neubau der Schicht und aller darunterliegenden an

14.5 ★ Non-Root-Ausführung

⚠ **In der Standardkonfiguration werden ALLE Prozesse im Container unter dem root -Account ausgeführt.** Ein Ausbruch aus einem Containerprozess mit root-Rechten kann **kritische Auswirkungen** haben, da **ein Containerprozess gleichberechtigt zu einem nativen Prozess** ist.

```
RUN addgroup -S spring && adduser -S springuser -G spring
USER springuser
```

⚠ **Neue Benutzer gehören standardmäßig der Gruppe root an** — deshalb muss **manuell eine andere Gruppe** angelegt werden.

14.6 📝 Hands-On: Dockerfile-Optimierung

✗ Vorher	✓ Nachher
FROM ubuntu	FROM openjdk:8-jre-alpine3.9 ← kleineres Image, präziser Tag
RUN apt-get update RUN apt-get install openjdk-8-jdk -y RUN apt-get install curl -y	RUN apk update && apk add curl ← Bündelung, Java schon im Baseimage
	RUN mkdir app RUN addgroup -S spring && adduser -S springuser -G spring USER springuser ← non-root
RUN mkdir app	WORKDIR app
COPY target/* app/	COPY target/app.jar . ← explizites COPY-Statement
CMD java -jar app.jar	CMD ["java", "-jar", "app.jar"] ← Exec-Notation für Sicherheit

📝 Aufgabe 7 mit Musterlösung

a) Optimierung

❌ Vorher	✅ Nachher	Begründung
FROM javaee/springdemo:103	FROM openjdk:8-jre-alpine3.9	Kleineres, offizielles , explizites Baseimage
ENV JAR_FILE=target/*.jar	ARG JAR_FILE=target/app.jar	Explizite Quelldatei , Auslagerung als ARG (nur Bauzeit)
ENV USER=dhbw-rv ENV PWD=adw123fde234awe	(entfällt) ENV MX_RAM=1024m	⚠️ Keine Nutzung von ENV für sensible Daten!
RUN apk update RUN apk add curl	RUN apk update && apk add curl	Bündelung → unnötige Schichten sparen
RUN mkdir app	RUN mkdir app RUN addgroup -S spring && adduser -S springuser -G spring USER springuser WORKDIR app	Non-Root-Ausführung
COPY \${JAR_FILE} app.jar	COPY \${JAR_FILE} app.jar	
CMD java -jar /app.jar - Xmx1024m	CMD ["java", "-jar", "app.jar", "- Xmx\${MX_RAM}"]	Exec-Notation , Speicherlimit als Umgebungsvariable

Build: `docker image build -t test:1 .` **Versionierung:** Beim **Baseimage** sollte ein **stable Tag** zum Einsatz kommen, um Updates zu bekommen und den Bauprozess deterministisch zu machen.

b) Warum ist ein SSH-Server im Container nicht sinnvoll?

1. Es gibt **explizite Docker-Befehle** (`docker container run / start` mit `-i`), die eine direkte Befehlseingabe bieten
2. Ein SSH-Server bedarf **Updateaufwand** und bietet **Angriffsfläche**
3. Ein SSH-Server bedarf **Administrationsaufwand** (Benutzer- und Passwort-Management)
4. **Dynamische horizontale Skalierung ist nicht kompatibel** mit der Nutzung eines statischen SSH-Servers

c) Kann man `mvn clean install` ins Dockerfile aufnehmen?

Rein technisch möglich (als RUN-Befehl), aber der Befehl wird dann **im Container ausgeführt und nicht auf dem Host**. Für die Nutzung von Dockerfiles zum **Compilieren und Packen** einer Anwendung sollten **multi-stage Dockerfiles** eingesetzt werden.



Aufgabe 8 mit Musterlösung — Dockerfile aus Anforderungen

```
# - offizielles 'openjdk' Baseimage, Zweig '16-jdk-alpine'
FROM openjdk:16-jdk-alpine

# - Anwendung horcht im Container auf Port 8080
EXPOSE 8080

# - 'apk update' + 'apk add vim -y' → GEBÜNDELT
RUN apk update && apk add vim -y

# - neues Verzeichnis /usr/src/app
RUN mkdir -p /usr/src/app

# - neuer Benutzer javauser + Gruppe 'java' mit eingeschränkten Rechten
RUN addgroup -S java && adduser -S javauser -G java
USER javauser

# - jar-Datei aus target nach /usr/src/app kopieren
COPY target/app.jar /usr/src/app/app.jar

# - Start in EXEC-Notation
CMD ["java", "-jar", "/usr/src/app/app.jar"]
```



mvn clean install wurde bewusst NICHT aufgenommen (siehe Aufgabe 7c) — der Hostport 3030 wird über `-p 3030:8080` beim `docker container run` gemappt, **nicht** im Dockerfile.



PROBEKLAUSUREN

Probeklausur 1 — Aufbau

Modul: Neue Konzepte · **Veranstaltung:** Cloud Computing · **Dauer:** 60 Minuten · **Max. Punkte:** 50 · **Hilfsmittel:** keine

Aufgabe	Thema	Punkte
1	Cloud-Computing-Grundlagen	17
2	Virtualisierung	16
3	Docker	17
Σ		50

Aufgabe 1 — Cloud-Computing-Grundlagen (17 P)

Teil	Frage	P
a	Was sind „ Server-Festungen “, wodurch entstehen sie und warum sind sie ein Problem für den Betrieb?	3
b	Wodurch entstehen lange Bereitstellungszeiten beim traditionellen On-Premise-Betrieb und was sind die gewollten Auswirkungen hiervon?	2
c	Nennen und erläutern Sie 4 Vorteile des Cloud-Computing	4
d	Nennen Sie 3 Herausforderungen des Cloud-Computing und erläutern Sie deren Ursachen	3
e	Erläutern Sie Continuous Integration, Continuous Delivery und Continuous Deployment und grenzen Sie die Begriffe voneinander ab	3
f	Erläutern Sie das Prinzip „ mutable infrastructure “	2

Aufgabe 2 — Virtualisierung (16 P)

Teil	Frage	P
a	Benennen Sie die 7 Abstraktionsebenen der Schichtenarchitektur	3, 5
b	Ordnen Sie jeder Dienstleistungsform das konkret verwendete Nutzungsobjekt der Schichtenarchitektur zu	2, 5
c	Was sind Vorteile des geschichteten Aufbaus , angesichts dessen, dass jede Schicht zusätzlichen Wartungsaufwand verursacht?	2
d	Welche Dienstleistungsstufe hat Parallelen zum Mieten eines Autos und warum? (→ <i>CaaS</i>)	2
e	Erläutern Sie zwei Vorteile und zwei Herausforderungen einer Hybrid Cloud	4
f	Erläutern Sie den Unterschied zwischen Edge Computing und einer Private Cloud in Bezug auf den Anwendungsfokus	2

Aufgabe 3 — Docker (17 P)

Gegeben: Spring-Boot-Anwendung, Port 8080, RestController mit `@Profile("dev")`, `app.jar` im aktuellen Ordner.

Teil	Frage	P
a	Überführen Sie 7 Anweisungen in Dockerfile-Befehle	3, 5
b	Erläutern Sie zwei Optimierungen des erstellten Dockerfiles mit Zeilenverweis	2
c	Warum wird <code>sayHello</code> beim Aufruf von <code>http://localhost:8080/hello?forename=taylor</code> nicht aufgerufen? (Hinweis: <code>@Profile("dev")</code> vs. <code>ENV Spring.profiles.active=prod</code>)	2
d	Änderungen, um eine Liste <code>'Check', 'Siri', 'CoolAid'</code> im Dockerfile zu definieren und in <code>specialNames</code> zu laden	2, 5
e	Dockerbefehl zum Bauen mit Tag <code>exam:1</code> → <code>docker image build -t exam:1 .</code>	1
f	Dockerbefehl zur Ausgabe der Schichten → <code>docker image history exam:1</code>	1
g	Welches Problem kann durch diesen Versionstag in Kombination mit einem Orchestrator auftreten?	2
h	Erläutern Sie zwei Strategien für unique Tags (→ <i>BuildId, Datum</i>)	1
i	Erläutern Sie das Prinzip von Copy-On-Write	2

Aufgabe a) — Lösungsansatz

```
FROM javaee/springdemo:latest
ENV Spring.profiles.active=prod
RUN mkdir app
WORKDIR app
COPY . .
EXPOSE 8080
CMD ["java", "-jar", "app.jar"]
```

Probeklausur 2 — Aufgabe 4 (10 P)

Spring-Boot-Anwendung, Port 8000.

a) Anweisungen → Dockerfile (2 P)

```
FROM openjdk:16-jdk-alpine
COPY target .
ENV stage=prod
CMD ["java", "-jar", "app.jar"]
```

b) Welcher Befehl fehlt? (1 P) → `EXPOSE 8000`

c) Abgeändertes Dockerfile (4 P)

```
FROM openjdk:16-jdk-alpine
ARG stage=prod                                # nur Bauzeit!
RUN mkdir app
WORKDIR app
RUN addgroup -S javagr && adduser -S java -G javagr
USER java
COPY target/app.jar .                          # nur app.jar!
EXPOSE 8000
CMD ["java", "-jar", "app.jar"]
```

d) Zwei Docker-Optionen für sensible Umgebungsvariablen beim `docker container run` (1 P) → `--env` und `--env-file`

SELBSTTEST — Fragen zum Abfragen

▼ Kapitel 1: Grundlagen

1. Definiere On-Premise. Welche Risiken trägt der Kunde, welchen Vorteil hat er?
2. Erkläre die Wolken-Analogie und nenne die RedHat-Definition von Clouds.
3. Nenne vier Kernmerkmale von Cloud Computing.
4. Welche vier Bereiche verändert Cloud Computing?
5. **Was sind Server-Festungen, wodurch entstehen sie, warum sind sie ein Problem?**
6. Was ist der Schneeflocken-Effekt (Martin Fowler)? Welche drei Folgen hat er?
7. Wodurch entstehen lange Bereitstellungszeiten beim On-Premise-Betrieb? Welche vier Wirkungen hat die hohe Kapitalbindung?
8. Warum führt „notwendige Auslastung“ zu Problemen?
9. Warum werden Wartungsaktivitäten aufgeschoben und welche Risiken entstehen dadurch?
10. Was ist die „Verteidigung des Status Quo“ und welche drei Wirkungen hat sie?
11. Nenne die sechs Vorteile von Cloud Computing.
12. Definiere Skalierung und Skalierungsfaktor. Nenne je ein Beispiel für linear und exponentiell.
13. **Vertikale vs. horizontale Skalierung: Definition, je 2 Vor- und Nachteile.**
14. Was war das Amazon-Problem und was folgte daraus?
15. Nenne vier innovative Cloud-Funktionsbausteine.
16. Warum ist globale Verteilung wichtig? Nenne vier Anwendungsfälle.
17. Was ändert On-Demand am Bereitstellungsprozess? Nenne drei Folgen.
18. Erkläre Skaleneffekte und Fixkostendegression. Nenne die drei Dimensionen mit je zwei Beispielen.
19. Vergleiche die Kostenstruktur On-Premise vs. Cloud (je vier Punkte).
20. Was ist Fixkostenvariabilisierung? Was ist der wichtigste Vorteil?
21. **Warum ist der reine Kostenvergleich irreführend?**
22. Rechne die monatlichen On-Premise-Kosten für einen Server aus (Hardware/Strom/Personal).
23. Nenne die vier Herausforderungen von Cloud Computing.
24. Nenne die vier Ursachen für Kostenüberschreitungen. Welche zwei sind empirisch bestätigt?
25. Nenne vier Sicherheitsrisiken der Cloud mit Erklärung.
26. Nenne die drei Probleme fehlender IT-Governance.
27. **Mutable vs. Immutable Infrastructure: Vorgehen, Dokumentation, Folge, Werkzeuge.**
28. Was ist der Grundgedanke der Immutable Infrastructure (6 Minuten vs. 6 Jahre)?
29. **Erkläre und grenze ab: Continuous Integration, Continuous Delivery, Continuous Deployment.**
30. Warum sind diese Automatisierungen bei horizontaler Skalierung notwendig?

▼ Kapitel 2: Virtualisierung

31. Definiere Virtualisierung. Was bleibt trotz Entkopplung bestehen?
32. **Grenze Simulation, Emulation und Virtualisierung ab (Definition, Wirkzusammenhänge, Praxiseinsatz, Beispiele).**
33. Warum kann man x86 nicht auf ARM virtualisieren — und was macht Rosetta 2 stattdessen?
34. Nenne die drei Gründe für Virtualisierung mit je drei Unterpunkten.
35. **Nenne die vier Schichten eines Computersystems von unten nach oben mit Funktion.**
36. Erkläre die Voll-Virtualisierung. Welche Rolle hat der Hypervisor genau?
37. **Typ-I- vs. Typ-II-Hypervisor: Position, Performance, Sicherheit, Beispiele.**
38. Was bedeutet „Para“ bei der Para-Virtualisierung? Wie funktioniert sie?
39. Was ist der entscheidende Nachteil der Para-Virtualisierung?
40. Erkläre die Prozessvirtualisierung. **Was machen cgroups, was machen namespaces?**
41. Nenne den zentralen Vorteil und den zentralen Nachteil der Prozessvirtualisierung.
42. **Vergleiche VM und Container entlang aller 9 Kriterien (Ressourcennutzung, Isolation, Applikationsstart, Ressourcenbereitstellung, Performance, Entkopplung, Imagegröße, horizontale Skalierbarkeit, Distributionseinheit).**
43. Wann VM, wann Container? Nenne je 5 Do's und Don'ts.
44. Nenne fünf Vorteile der Kombination von Voll- und Prozessvirtualisierung.
45. **Nenne sieben Gründe für Overhead bei VMs und den einen bei Containern.**
46. Nenne sieben Vorteile der Prozessvirtualisierung für Entwicklung, Test, Betrieb.
47. **Nenne die 7 Abstraktionsebenen der Cloud-Schichtenarchitektur.**
48. In welchem Verhältnis stehen Kontrolle und Abstraktion? Was ist die Gefahr?
49. **Beschreibe IaaS, CaaS, PaaS, FaaS, SaaS mit Auto-Analogie, Nutzungsobjekt, Abstraktion und Lock-In-Grad.**
50. Welche Dienstleistungsstufe entspricht dem Mietauto und warum?
51. Ab welcher Stufe ist der Kunde nicht mehr Fahrer, sondern Beifahrer?
52. Was gibt man mit der Kontrolle NICHT nur ab, sondern auch?
53. Nenne für jedes der fünf Modelle drei Einsatzgebiete plus ein Praxisbeispiel.
54. **Beschreibe die fünf Bereitstellungsmodelle mit je 2–3 Vor- und Nachteilen.**
55. Was ist eine Airgap-Umgebung?
56. Wo verlagert sich der Lock-In bei der Multi Cloud hin?
57. Wie erreicht man bei einer VPC die Isolation?
58. **Ordne die fünf Bereitstellungsmodelle nach Wartungsaufwand und Abstraktion ein.**
59. **Nenne die drei Migrationspfade (Start → Zwischenziel → Endziel) je nach Ausgangslage.**
60. Definiere Edge Computing. Erkläre Far Edge / Edge / Cloud am Gießerei-Beispiel.
61. Nenne sechs Anwendungsfelder von Edge Computing mit Beispiel.
62. **Grenze Edge Computing und Private Cloud entlang von sechs Kriterien ab.**
63. Was sind die beiden Herausforderungen des Edge Computing?

▼ Kapitel 3: Docker

64. Erkläre das Frachtcontainer-Paradigma (4 Punkte vorher/nachher) und die 18-Mann-/9-Mann-Zahlen.
65. Was ist Docker? Nenne die beiden Bestandteile mit Funktion.
66. Zeichne den Docker-Workflow (Dockerfile / Image / Container / Registry) mit den Befehlen.
67. Erkläre die Kuchenrezept-Analogie.
68. **Image vs. Container: je vier Merkmale. Was ist der Kontrast zur VM?**
69. **Erkläre Copy-on-Write. Was passiert beim Stoppen, was beim Löschen eines Containers?**
70. Zerlege `docker container run -d hello-world` in seine fünf Bestandteile.
71. Nenne die sechs wichtigsten Container-Befehle mit Wirkung.
72. Was ist der Unterschied zwischen `docker container run` und `docker container start`?
73. Erkläre `-p`, `-d`, `-it` und `--rm`.
74. Beschreibe die sieben Schritte bei `docker container run hello-world`.
75. Nenne fünf typische Deployment-Fehlerursachen.
76. **Erkläre die Schichtenarchitektur eines Images und ihre vier Vorteile.**
77. Was passiert bei Änderung einer mittleren Schicht?
78. **Zerlege einen Docker-Tag in seine vier Abschnitte. Welche sind Pflicht?**
79. **Warum ist `latest` gefährlich? Nenne vier Gründe.**
80. Was ist eine Docker Registry? public vs. private? Woran erkennt man offizielle Images?
81. Was ist ein Dockerfile? Wie heißt es? Wie ist es dreigeteilt? Wie lautet der Build-Befehl?
82. **Erkläre FROM, ENV, ARG, RUN, WORKDIR, COPY, EXPOSE, USER, CMD.**
83. **Was ist der Unterschied zwischen EXPOSE und der publish-Option?**
84. **Exec form vs. Shell form bei CMD — was ist der Sicherheitsunterschied?**
85. Nenne die sechs wichtigsten Image-Befehle.
86. Wozu dienen Umgebungsvariablen und Profile in Spring? Was macht `mvn clean install`?
87. **Baseimage Best Practices: vier Regeln. Nenne die Größen von ubuntu, opensuse, alpine.**
88. **Warum müssen Images bei Baseimage-Updates neu gebaut werden?**
89. Stable Tags vs. Unique Tags: wofür jeweils? Warum keine Git-Tags?
90. **Warum dürfen keine Passwörter in ENV? Wie macht man es richtig?**
91. Nenne die Lesereihenfolge von Umgebungsvariablen.
92. **Nach welchem Prinzip ordnet man Befehle im Dockerfile an? Warum ist ein frühes COPY fatal?**
93. Wie vermeidet man unnötige Schichten?
94. **Warum ist Non-Root wichtig? Wie legt man einen unprivilegierten User an — und was ist die Falle?**
95. Optimierte ein Dockerfile mit `FROM ubuntu`, drei RUN-Befehlen, Wildcard-COPY und Shell-CMD.
96. **Warum ist ein SSH-Server im Container nicht sinnvoll? Nenne vier Gründe.**
97. **Kann `mvn clean install` ins Dockerfile? Was ist die Konsequenz? Was ist die Lösung?**
98. Schreibe ein vollständiges Best-Practice-Dockerfile aus einer Anforderungsliste.



Materialien im Modul

Datei	Inhalt
1-Grundlagen.pdf	Kapitel 1 (38 Folien mit Sprechernotizen)
2-Virtualisierung.pdf	Virtualisierung, Dienstleistungs- und Bereitstellungsmodelle, Edge Computing (36 Folien)
3-Docker.pdf	Docker Container, Images, Best Practices (45 Folien)
Aufgabe_1–8_Lösung.pdf	Musterlösungen zu allen Übungen
Aufgabe_6.zip / Aufgabe_6_Lösung.zip	Spring-Boot-Projekt (ApiKey-Authentifizierung, Spring Profiles)
Probeklausur.pdf / Probeklausur_2.pdf	Zwei Probeklausuren
Azure Preisrechner	https://azure.microsoft.com/de-de/pricing/calculator/
Hands-On-Umgebungen	killercode.com/playgrounds/scenario/kubernetes